

PARTNYOR O'TKAZMALARINI KO'RSATISH TIZIMI

Texnik hujjat (TZ)

Backend: .NET 10 (ASP.NET Core Web API)

Frontend: Vue 3

Ma'lumotlar bazasi: PostgreSQL + EF Core

Sana: 11.06.2026

Mundarija

- 1. Loyiha haqida
- 2. Texnologiyalar steki
- 3. Tizim arxitekturası
- 4. Rollar va autentifikatsiya
- 5. Funksional talablar (jadval va modal)
- 6. Ma'lumotlar bazasi (SQL)
- 7. Backend: .NET 10 modellar va EF Core
- 8. Backend: API endpointlar va JWT
- 9. Frontend: Vue 3 (struktura va komponentlar)
- 10. Bajarilish ketma-ketligi
- 11. Xavfsizlik talablari
- 12. Loyihani ishga tushirish

1. Loyiha haqida

Tizimning vazifasi - partnyorlar (Payme, Click, Uzum va boshqalar) orqali amalga oshirilgan pul o'tkazmalarini foydalanuvchilarga ko'rsatish. O'tkazmalar tayyor bazada saqlanadi; tizim ularni o'qib, foydalanuvchining roli asosida filtrlab, jadval va modal ko'rinishida namoyish etadi.

Asosiy talablar

- Foydalanuvchi login va parol bilan tizimga kiradi.
- Foydalanuvchiga berilgan rol orqali faqat kerakli partnyor ma'lumotlari ko'rsatiladi.
- O'tkazmalar jadval ko'rinishida chiqadi (5 ta asosiy ustun).
- Jadval qatoriga bosilganda modal ochilib, o'tkazmaning to'liq tafsilotlari (passport, karta, summa, xato) ko'rsatiladi.

2. Texnologiyalar steki

Qatlam	Texnologiya
Backend	.NET 10 - ASP.NET Core Web API (C#)
ORM	Entity Framework Core 10
Ma'lumotlar bazasi	PostgreSQL (Npgsql provayder)
Autentifikatsiya	JWT Bearer + BCrypt (parol hash)
Frontend	Vue 3 (Composition API) + Vite
HTTP klient	Axios
State (holat)	Pinia
Routing	Vue Router
UI kutubxona	(taklif) Element Plus yoki PrimeVue

3. Tizim arxitekturası

Umumiy oqim: foydalanuvchi Vue 3 ilovasiga kiradi, .NET API'ga so'rov yuboradi, API JWT tokenni tekshirib, rol bo'yicha bazadan ma'lumotni filtrlab qaytaradi.

```
+-----+                                     +-----+
```

Vue 3 (SPA)	----->	.NET 10 API
- Login sahifa	1. login -> JWT token	- AuthController
- Jadval	2. GET /payments (token)	- Payments Ctrl
- Modal	<-----	- JWT middleware

```
+-----+                                     +-----+
```

ma'lumot (rol bo'yicha)

|
EF Core | SQL
v

```
+-----+
```

PostgreSQL
roles, partners
users, payments

```
+-----+
```

4. Rollar va autentifikatsiya

Ro'l	Tavsif	Ko'radigan ma'lumot
admin	Tizim administratori	Barcha partnyorlar o'tkazmalari
partner	Partnyor xodimi	Faqat o'ziga biriktirilgan partnyor

Kirish jarayoni

- Foydalanuvchi login va parolni yuboradi.
- Server parolni BCrypt hash bilan solishtiradi.
- To'g'ri bo'lsa JWT token beriladi (ichida userId, role, partnerId).
- Har bir keyingi so'rovda token tekshiriladi.

MUHIM (xavfsizlik): partner roli uchun SQL so'rovga server tomonidan majburiy partnerId sharti qo'shiladi. PartnerId tokenidan olinadi, frontenddan kelgan qiymatga ishonilmaydi - shu sababli foydalanuvchi boshqa partnyor ma'lumotini hech qachon ko'ra olmaydi.

5. Funksional talablar

5.1. O'tkazmalar ro'yxati (jadval)

Ustun (ekranda)	Manba maydoni	Izoh
To'lov identifikatori	PaymentRef	Unikal kod
O'tkazma qatnashchilari	Sender -> Receiver	Jo'nat. / qabul q.
O'tkazma holati	Status	success/pending/...
O'tkazma sanasi	TransferDate	Sana va vaqt
O'tkazma summasi (so'mda)	AmountUzs	150 000 so'm

- Sahifalash (pagination).
- Sana oralig'i, holat va PaymentRef bo'yicha filtr/qidiruv.
- Holat rang bilan ajratiladi (yashil/sariq/qizil).

5.2. O'tkazma tafsilotlari (modal)

Jadval qatoriga bosilganda modal ochiladi va quyidagilar ko'rsatiladi:

Jo'natuvchi va Qabul qiluvchi

- F.I.Sh.
- Passport ma'lumotlari (seriya + raqam).
- Karta raqami (maskalangan: 8600 **** * 1234).

Summa tafsilotlari

- O'tkazilgan summa (asl valyutada).
- So'mga o'girilgan summa.
- Dollar kursi.
- Komissiya.

Xato (agar mavjud bo'lsa)

- Xato kodi va xato matni qizil panelda ko'rsatiladi.

6. Ma'lumotlar bazasi (SQL)

To'rtta jadval: roles, partners, users, payments. Quyida PostgreSQL sxemasi keltirilgan.

```
CREATE TABLE roles (  
    id          SERIAL PRIMARY KEY,  
    name        VARCHAR(50) UNIQUE NOT NULL,    -- 'admin' | 'partner'  
    description VARCHAR(255)  
);  
  
CREATE TABLE partners (  
    id          SERIAL PRIMARY KEY,  
    name        VARCHAR(100) NOT NULL,  
    code        VARCHAR(50) UNIQUE NOT NULL,    -- payme, click, uzum  
    is_active   BOOLEAN DEFAULT true  
);  
  
CREATE TABLE users (  
    id          SERIAL PRIMARY KEY,  
    login       VARCHAR(100) UNIQUE NOT NULL,  
    password_hash VARCHAR(255) NOT NULL,  
    full_name   VARCHAR(150),  
    role_id     INT NOT NULL REFERENCES roles(id),  
    partner_id  INT REFERENCES partners(id),    -- admin uchun NULL  
    is_active   BOOLEAN DEFAULT true  
);  
  
CREATE TABLE payments (  
    id          BIGSERIAL PRIMARY KEY,  
    payment_ref VARCHAR(100) UNIQUE NOT NULL,    -- PaymentRef  
    partner_id  INT NOT NULL REFERENCES partners(id),  
    status      VARCHAR(30) NOT NULL,            -- success/pending/failed  
    transfer_date TIMESTAMP NOT NULL,  
    amount_uzs  NUMERIC(18,2) NOT NULL,         -- summa (so'mda)  
    -- Jo'natuvchi  
    sender_full_name    VARCHAR(200),  
    sender_passport_series VARCHAR(10),  
    sender_passport_number VARCHAR(20),  
    sender_card_number  VARCHAR(25),  
    -- Qabul qiluvchi  
    receiver_full_name  VARCHAR(200),  
    receiver_passport_series VARCHAR(10),  
    receiver_passport_number VARCHAR(20),  
    receiver_card_number VARCHAR(25),  
    -- Summa tafsilotlari  
    amount_original  NUMERIC(18,2),    -- asl valyutadagi summa  
    original_currency VARCHAR(3),      -- USD, RUB ...  
    usd_rate         NUMERIC(12,4),    -- dollar kursi  
    commission       NUMERIC(18,2),    -- komissiya  
    -- Xato  
    error_code       VARCHAR(50),  
    error_message    TEXT,
```

```

        created_at    TIMESTAMP DEFAULT now()
    );

    CREATE INDEX idx_payments_partner ON payments(partner_id);
    CREATE INDEX idx_payments_date    ON payments(transfer_date);
    CREATE INDEX idx_payments_status  ON payments(status);

```

7. Backend: .NET 10 modellari va EF Core

Loyiha tuzilishi (Clean / Layered yondashuv):

PartnerPayments.Api/

```

|- Models/           (Entity klasslar: Role, Partner, User, Payment)
|- Data/             (AppDbContext)
|- Dtos/             (PaymentListDto, PaymentDetailDto, LoginDto)
|- Services/         (AuthService, PaymentService)
|- Controllers/      (AuthController, PaymentsController)
|- Program.cs        (DI, JWT, CORS sozlamalari)
|- appsettings.json

```

Entity: Payment.cs

```

namespace PartnerPayments.Api.Models;

public class Payment
{
    public long Id { get; set; }
    public string PaymentRef { get; set; } = default!;
    public int PartnerId { get; set; }
    public Partner? Partner { get; set; }
    public string Status { get; set; } = "pending";
    public DateTime TransferDate { get; set; }
    public decimal AmountUzs { get; set; }

    // Jo'natuvchi
    public string? SenderFullName { get; set; }
    public string? SenderPassportSeries { get; set; }
    public string? SenderPassportNumber { get; set; }
    public string? SenderCardNumber { get; set; }

    // Qabul qiluvchi
    public string? ReceiverFullName { get; set; }
    public string? ReceiverPassportSeries { get; set; }
    public string? ReceiverPassportNumber { get; set; }
    public string? ReceiverCardNumber { get; set; }

    // Summa tafsilotlari
    public decimal? AmountOriginal { get; set; }
    public string? OriginalCurrency { get; set; }
    public decimal? UsdRate { get; set; }
    public decimal? Commission { get; set; }

    // Xato
    public string? ErrorCode { get; set; }
    public string? ErrorMessage { get; set; }
    public DateTime CreatedAt { get; set; }
}

```

Entity: User, Role, Partner

```
public class Role
{
    public int Id { get; set; }
    public string Name { get; set; } = default!; // admin | partner
    public string? Description { get; set; }
}

public class Partner
{
    public int Id { get; set; }
    public string Name { get; set; } = default!;
    public string Code { get; set; } = default!;
    public bool IsActive { get; set; } = true;
}

public class User
{
    public int Id { get; set; }
    public string Login { get; set; } = default!;
    public string PasswordHash { get; set; } = default!;
    public string? FullName { get; set; }
    public int RoleId { get; set; }
    public Role? Role { get; set; }
    public int? PartnerId { get; set; } // admin uchun null
    public Partner? Partner { get; set; }
    public bool IsActive { get; set; } = true;
}
```

AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using PartnerPayments.Api.Models;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> o) : base(o) { }

    public DbSet<Role> Roles => Set<Role>();
    public DbSet<Partner> Partners => Set<Partner>();
    public DbSet<User> Users => Set<User>();
    public DbSet<Payment> Payments => Set<Payment>();

    protected override void OnModelCreating(ModelBuilder b)
    {
        b.Entity<Payment>().HasIndex(p => p.PaymentRef).IsUnique();
        b.Entity<Payment>().HasIndex(p => p.PartnerId);
        b.Entity<User>().HasIndex(u => u.Login).IsUnique();
        b.Entity<Payment>().Property(p => p.AmountUzs).HasPrecision(18, 2);
    }
}
```

8. Backend: API endpointlar va JWT

+-----+-----+-----+-----+-----+-----+

Metod	URL	Rol	Tavsif
POST	/api/auth/login	hammasi	Login -> JWT token
GET	/api/payments	admin, partner	Ro'yxat (filtr+page)
GET	/api/payments/{id}	admin, partner	Modal tafsiloti
GET	/api/partners	admin	Filtr uchun ro'yxat

DTO klasslar

```
public record LoginDto(string Login, string Password);

public record PaymentListDto(long Id, string PaymentRef,
    string Participants, string Status, DateTime TransferDate,
    decimal AmountUzs);

public record AmountDto(decimal? Original, string? Currency,
    decimal Uzs, decimal? UsdRate, decimal? Commission);

public record PartyDto(string? FullName, string? Passport,
    string? CardNumber);
```

PaymentsController.cs (rol bo'yicha filtr)

```
[ApiController]
[Route("api/payments")]
[Authorize]
public class PaymentsController : ControllerBase
{
    private readonly AppDbContext _db;
    public PaymentsController(AppDbContext db) => _db = db;

    [HttpGet]
    public async Task<IActionResult> List(int page = 1, int limit = 20,
        string? status = null, string? search = null)
    {
        var q = _db.Payments.AsQueryable();

        // Rol bo'yicha cheklov: partner faqat o'z ma'lumotini ko'radi
        var role = User.FindFirst("role")?.Value;
        if (role == "partner")
        {
            var pid = int.Parse(User.FindFirst("partnerId")!.Value);
            q = q.Where(p => p.PartnerId == pid);
        }

        if (!string.IsNullOrEmpty(status))
            q = q.Where(p => p.Status == status);
        if (!string.IsNullOrEmpty(search))
            q = q.Where(p => p.PaymentRef.Contains(search));

        var total = await q.CountAsync();
        var items = await q.OrderByDescending(p => p.TransferDate)
            .Skip((page - 1) * limit).Take(limit)
            .Select(p => new PaymentListDto(p.Id, p.PaymentRef,
                (p.SenderFullName ?? "-") + " -> " +
                (p.ReceiverFullName ?? "-"),
                p.Status, p.TransferDate, p.AmountUzs))
```



```

        .ToListAsync();

        return Ok(new { data = items,
            pagination = new { page, limit, total } });
    }

    [HttpGet("{id:long}")]
    public async Task<IActionResult> Detail(long id)
    {
        var p = await _db.Payments.FindAsync(id);
        if (p is null) return NotFound();

        var role = User.FindFirst("role")?.Value;
        if (role == "partner")
        {
            var pid = int.Parse(User.FindFirst("partnerId")!.Value);
            if (p.PartnerId != pid) return Forbid();
        }

        return Ok(new {
            payment_ref = p.PaymentRef, status = p.Status,
            transfer_date = p.TransferDate,
            sender = new PartyDto(p.SenderFullName,
                $"{p.SenderPassportSeries} {p.SenderPassportNumber}",
                Mask(p.SenderCardNumber)),
            receiver = new PartyDto(p.ReceiverFullName,
                $"{p.ReceiverPassportSeries} {p.ReceiverPassportNumber}",
                Mask(p.ReceiverCardNumber)),
            amount = new AmountDto(p.AmountOriginal, p.OriginalCurrency,
                p.AmountUzs, p.UsdRate, p.Commission),
            error = p.ErrorCode == null ? null :
                new { code = p.ErrorCode, message = p.ErrorMessage }
        });
    }

    private static string? Mask(string? c)
    {
        if (string.IsNullOrEmpty(c) || c.Length < 8) return c;
        return c[..4] + " **** " + c[^4..];
    }
}

```

AuthController.cs (login + JWT)

```

[ApiController]
[Route("api/auth")]
public class AuthController : ControllerBase
{
    private readonly AppDbContext _db;
    private readonly IConfiguration _cfg;
    public AuthController(AppDbContext db, IConfiguration cfg)
    {
        _db = db; _cfg = cfg;
    }

    [HttpPost("login")]
    public async Task<IActionResult> Login(LoginDto dto)
    {
        var user = await _db.Users.Include(u => u.Role)

```

```

        .FirstOrDefaultAsync(u => u.Login == dto.Login && u.IsActive);
    if (user is null ||
        !BCrypt.Net.BCrypt.Verify(dto.Password, user.PasswordHash))
        return Unauthorized(new { message = "Login yoki parol xato" });

    var claims = new List<Claim> {
        new("userId", user.Id.ToString()),
        new("role", user.Role!.Name),
        new("partnerId", user.PartnerId?.ToString() ?? "")
    };
    var key = new SymmetricSecurityKey(
        Encoding.UTF8.GetBytes(_cfg["Jwt:Key"]!));
    var creds = new SigningCredentials(key,
        SecurityAlgorithms.HmacSha256);
    var token = new JwtSecurityToken(claims: claims,
        expires: DateTime.UtcNow.AddHours(8), signingCredentials: creds);
    return Ok(new {
        token = new JwtSecurityTokenHandler().WriteToken(token),
        role = user.Role.Name });
}
}

```

Program.cs (JWT + CORS sozlamasi - qisqacha)

```

builder.Services.AddDbContext<AppDbContext>(o =>
    o.UseNpgsql(builder.Configuration.GetConnectionString("Default")));

builder.Services.AddAuthentication("Bearer")
    .AddJwtBearer(o => o.TokenValidationParameters = new()
    {
        ValidateIssuerSigningKey = true,
        IssuerSigningKey = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]!)),
        ValidateIssuer = false, ValidateAudience = false
    });

builder.Services.AddCors(o => o.AddDefaultPolicy(p =>
    p.WithOrigins("http://localhost:5173")
    .AllowAnyHeader().AllowAnyMethod()));

```

9. Frontend: Vue 3

Loyiha tuzilishi (Vite + Vue 3 Composition API):

```

frontend/
|- src/
|  |- api/          axios.js  (HTTP klient + token)
|  |- stores/       auth.js   (Pinia - login holati)
|  |- views/        LoginView.vue, PaymentsView.vue
|  |- components/   PaymentModal.vue
|  |- router/       index.js
|  |- App.vue, main.js
|- vite.config.js

```

api/axios.js

```

import axios from 'axios'

```

```

const api = axios.create({ baseURL: 'http://localhost:5000/api' })

// Har bir so'rovga JWT tokenni qo'shamiz
api.interceptors.request.use(cfg => {
  const token = localStorage.getItem('token')
  if (token) cfg.headers.Authorization = `Bearer ${token}`
  return cfg
})

export default api

```

views/PaymentsView.vue (jadval)

```

<script setup>
import { ref, onMounted } from 'vue'
import api from '../api/axios'
import PaymentModal from '../components/PaymentModal.vue'

const items = ref([])
const selectedId = ref(null)
const search = ref('')

async function load() {
  const { data } = await api.get('/payments', {
    params: { search: search.value, page: 1, limit: 20 }
  })
  items.value = data.data
}

onMounted(load)

function fmt(n) { return new Intl.NumberFormat('uz-UZ').format(n) }
</script>

<template>
  <div class="toolbar">
    <input v-model="search" placeholder="PaymentRef qidirish..." />
    <button @click="load">Qidirish</button>
  </div>

  <table>
    <thead>
      <tr>
        <th>To'lov ID</th><th>Qatnashchilar</th><th>Holat</th>
        <th>Sana</th><th>Summa (so'm)</th>
      </tr>
    </thead>
    <tbody>
      <tr v-for="p in items" :key="p.id"
        @click="selectedId = p.id" class="row">
        <td>{{ p.paymentRef }}</td>
        <td>{{ p.participants }}</td>
        <td><span :class="p.status">{{ p.status }}</span></td>
        <td>{{ new Date(p.transferDate).toLocaleString() }}</td>
        <td>{{ fmt(p.amountUzs) }} so'm</td>
      </tr>
    </tbody>
  </table>

```

```
</table>
```

```
<PaymentModal v-if="selectedId" :id="selectedId"
  @close="selectedId = null" />
```

```
</template>
```

components/PaymentModal.vue (tafsilot)

```
<script setup>
```

```
import { ref, onMounted } from 'vue'
```

```
import api from '../api/axios'
```

```
const props = defineProps({ id: Number })
```

```
const emit = defineEmits(['close'])
```

```
const p = ref(null)
```

```
onMounted(async () => {
  const { data } = await api.get(`/payments/${props.id}`)
  p.value = data
})
```

```
</script>
```

```
<template>
```

```
<div class="overlay" @click.self="emit('close')">
```

```
<div class="modal" v-if="p">
```

```
<h3>0'tkazma: {{ p.payment_ref }}</h3>
```

```
<div class="cols">
```

```
<div>
```

```
<h4>Jo'natuvchi</h4>
```

```
<p>{{ p.sender.full_name }}</p>
```

```
<p>Passport: {{ p.sender.passport }}</p>
```

```
<p>Karta: {{ p.sender.card_number }}</p>
```

```
</div>
```

```
<div>
```

```
<h4>Qabul qiluvchi</h4>
```

```
<p>{{ p.receiver.full_name }}</p>
```

```
<p>Passport: {{ p.receiver.passport }}</p>
```

```
<p>Karta: {{ p.receiver.card_number }}</p>
```

```
</div>
```

```
</div>
```

```
<h4>Summa</h4>
```

```
<p>0'tkazilgan: {{ p.amount.original }} {{ p.amount.currency }}</p>
```

```
<p>So'mda: {{ p.amount.uzs }} | Kurs: {{ p.amount.usd_rate }}</p>
```

```
<p>Komissiya: {{ p.amount.commission }}</p>
```

```
<div v-if="p.error" class="error">
```

```
Xato [{{ p.error.code }}]: {{ p.error.message }}
```

```
</div>
```

```
<button @click="emit('close')">Yopish</button>
```

```
</div>
```

```
</div>
```

```
</template>
```

10. Bajarilish ketma-ketligi

#	Bosqich	Natija
1	.NET 10 Web API loyihasini yaratish	Skelet tayyor
2	EF Core + Npgsql ulash, Entity'lar	Modellar tayyor
3	Migration + Database update	Jadvallar yaratildi
4	Seed (admin, partner, test o'tkazmalar)	Sinov ma'lumoti
5	AuthController + JWT + BCrypt	Login ishlaydi
6	JWT middleware + rol cheklovi	Himoya o'rnatildi
7	PaymentsController: GET /payments	Ro'yxat API tayyor
8	GET /payments/{id} + maskalash	Modal API tayyor
9	Vue 3 loyihasi (Vite) + Pinia + Router	Frontend skeleti
10	Login sahifa + token saqlash	Kirish ishlaydi
11	Jadval (PaymentsView)	Ro'yxat ko'rinadi
12	Modal (PaymentModal) + format/rang	Tafsilot ko'rinadi
13	Test va deploy	Tayyor mahsulot

11. Xavfsizlik talablari

- Parollar faqat BCrypt hash ko'rinishida saqlanadi.
- Karta raqami va passport ma'lumotlari maskalanadi.
- Rol bo'yicha cheklov serverda (token claim'lari asosida) bajariladi.
- Barcha /api/payments so'rovlari [Authorize] bilan himoyalangan.
- EF Core parametrlangan so'rovlar -> SQL injection oldi olinadi.
- CORS faqat ishonchli frontend manzili uchun ochiladi.
- HTTPS majburiy, JWT tokenning amal qilish muddati cheklangan (8 soat).

12. Loyihani ishga tushirish

Backend (.NET 10)

```
dotnet new webapi -n PartnerPayments.Api
cd PartnerPayments.Api
dotnet add package Microsoft.EntityFrameworkCore
dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
dotnet add package BCrypt.Net-Next

# migration va baza
dotnet ef migrations add Init
dotnet ef database update
dotnet run
```

Frontend (Vue 3)

```
npm create vite@latest frontend -- --template vue
cd frontend
npm install
npm install axios pinia vue-router
npm run dev # http://localhost:5173
```

appsettings.json (namuna)

```
{
  "ConnectionStrings": {
    "Default": "Host=localhost;Port=5432;Database=payments_db;" +
      "Username=postgres;Password=postgres"
  },
  "Jwt": { "Key": "BU_YERGA_KUCHLI_MAXFIY_KALIT_QOYING_32+" }
}
```

--- Hujjat yakuni ---